

Split-Stem Active Depth Sensing for Slit Navigation

Anonymous Manuscript

Abstract—Differentiable simulation has made it possible to backpropagate task losses through dynamics, planning, and control, but the perception stage is still often treated as a non-differentiable boundary. As a result, gradient flow stops at the sensor output and the agent cannot directly optimize how it should sense the scene. We address this limitation with a differentiable active depth perception framework for quadrotor slit navigation, where the active depth sensor itself is modeled as part of the computational graph. Projector power, exposure, and gain become trainable variables that shape the observed depth map, allowing navigation loss to flow backward through sensing and control to the sensor parameters. The resulting closed loop does not only learn how to fly through a narrow aperture, but also how to configure the camera to preserve task-relevant geometry under glare, specular reflection, and low-reflectance degradation. To stabilize this learning process, we use a split-stem policy that keeps camera semantics fixed during final flight adaptation while still allowing the flight branch to improve. In held-out closed-loop evaluation, the final policy achieves 0.751 success, 0.249 collision, 0.975 depth fill, and 0.550 terminal distance, improving over fixed, random-fixed, non-differentiable learned-camera, and blind baselines. These results show that active depth sensing can complete the differentiable learning loop for embodied UAV navigation.

Index Terms—active sensing, depth camera, slit navigation, DAgger, differentiable perception, quadrotor navigation

I. Introduction

Differentiable robotics has recently expanded from dynamics and control into full simulation pipelines. Differentiable simulators now provide gradients through physical evolution, enabling robot policies to be trained with analytical feedback rather than pure trial-and-error search [1]–[4]. In parallel, camera simulation has begun to support inverse rendering and embodied AI with explicit optical modeling and calibration-aware rendering [5]. These advances have made the perception-to-control pipeline increasingly differentiable, but a key gap remains: the sensing stage itself is still often optimized as a fixed black box rather than as a trainable physical process.

This gap matters in vision-based UAV navigation, where the quality of a depth observation directly depends on the sensor configuration. Projector power, exposure, and gain are not merely image-style knobs. They determine whether the geometry needed for obstacle avoidance is visible at all. The issue becomes critical near narrow apertures, where glare, specular reflections, or weak return can erase the local depth support needed by the controller. In such cases, the policy may know that a maneuver failed, but it cannot propagate that failure back to the sensing parameters that caused it.

Prior work has already shown that perception and control should not be optimized independently. Perception-aware flight, trajectory planning, and speed adaptation

improve navigation by accounting for the sensing field of view or the state-estimation regime [6]–[10]. However, these methods still rely on fixed sensing models, hand-designed sensing rules, or teacher-student distillation. They do not provide a differentiable path from task loss to the physical parameters of the sensor itself.

In this paper, we close that loop with a differentiable active depth perception framework for slit navigation. The key idea is to model the active depth sensor as a differentiable module inside the sim-to-real training pipeline, so that the navigation loss can flow backward through sensing, simulation, and control to the sensor parameters. This enables the agent to learn not only how to fly through a narrow aperture, but also how to configure the camera so that the aperture geometry remains observable under degraded visual conditions. To keep this end-to-end learning stable, we further use a split-stem policy and a DAgger-style relabeling stage that separate camera semantics from final flight adaptation [11].

The main contributions of this paper are as follows.

- We propose a differentiable active depth perception model that makes projector power, exposure, and gain part of the computational graph.
- We integrate that model into a closed-loop UAV learning pipeline so that task gradients can propagate through perception, simulation, and control.
- We introduce a split-stem adaptation strategy and teacher relabeling procedure to stabilize the learned camera semantics during flight optimization.
- We validate the framework on a slit-navigation benchmark with glare, specular, and low-reflectance depth degradation regimes, showing improved closed-loop performance over fixed, random-fixed, non-differentiable, and blind baselines.

II. Related Work

A. Differentiable Simulation and Control

Differentiable simulation has become a core mechanism for closing the gap between physical dynamics and gradient-based policy learning [1]. In aerial robotics, this has enabled policies to be optimized directly through physics models, from diffusion-like differentiable control pipelines to vision-based agile flight [2]–[4], [12]. These studies establish the key premise that task loss can be propagated through dynamics and control, so that the learned policy is not tuned by heuristic reward shaping alone. However, their differentiability usually ends at the observation interface: the simulator can backpropagate through motion, but the sensing process is still treated

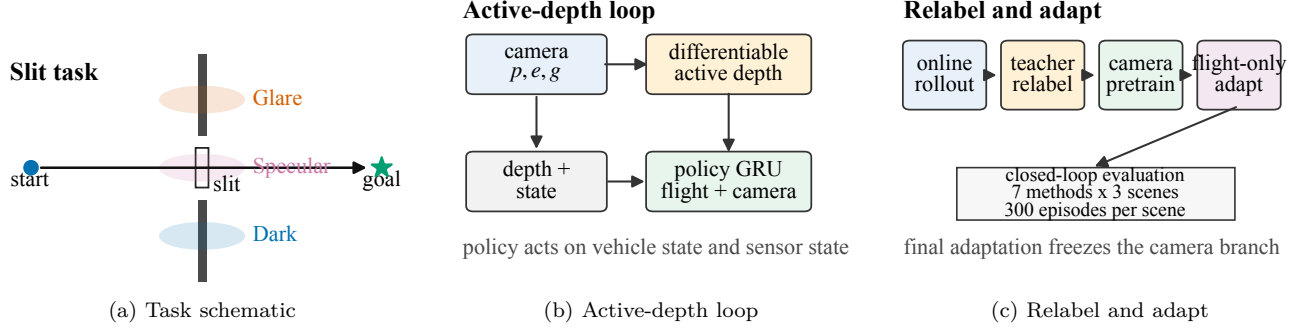


Fig. 1. Benchmark and training protocol. The task isolates active depth sensing in a single-wall slit-navigation environment with randomized aperture locations and localized sensing regimes. The policy controls both flight and camera parameters. Online states are relabeled by a differentiable camera teacher, the camera branch is pretrained on those labels, and final flight adaptation freezes the camera branch while updating the flight branch.

as an input generator rather than as a trainable physical module.

B. Differentiable Rendering and Camera Simulation

The second line of work concerns photorealistic simulation and inverse rendering. Procedural renderers and robotic simulators such as BlenderProc2, VISTA 2.0, Habitat 3.0, and Chrono::Sensor have made it easier to synthesize large-scale visual data for embodied AI [13]–[16]. In parallel, camera-simulation studies have examined how sensor parameters, optical artifacts, and calibration affect downstream perception [17]–[21]. DiffPhysCam extends this direction by making the camera itself differentiable and physically calibrated, so that optical effects such as defocus blur, exposure variation, and other sensor artifacts can be optimized together with the rest of the pipeline [5].

Recent differentiable-rendering systems further improve the fidelity of simulation-based learning. NeRF, Instant-NGP, Nerfstudio, Gaussian Splatting, SuGaR, and NVDiffRecMC have all advanced the reconstruction and synthesis of 3D scenes from images [22]–[27]. At the same time, several works model camera optics explicitly within the rendering loop, including Neural Camera Simulators, depth-of-field-aware differentiable rendering, and NeRF variants with editable camera settings or thin-lens effects [28]–[33]. These methods are highly relevant to our setting, but they mainly target photorealism or inverse rendering. By contrast, our objective is task-driven active sensing: the camera settings are optimized because they change the navigation outcome, not only because they improve image realism.

C. Perception-Aware and Active UAV Navigation

Vision-based UAV navigation has long recognized that perception and control must be coupled. Classical active-perception work established sensing as an action that should be chosen to improve future decisions [34]–[36]. Modern UAV systems follow the same principle in

more task-specific forms, including integrated perception-control, perception-aware agile flight, perception-aware trajectory planning, and speed adaptation in clutter [6]–[10]. Agile-flight platforms and benchmark systems further show how tightly coupled perception-action design improves high-speed flight [37]–[40]. These works motivate the present paper, but they still assume a fixed sensing model or a fixed observation encoder. Our contribution is to make the sensor parameters themselves differentiable and to use that gradient signal to close the loop between flight, sensing, and task loss.

D. Imitation Learning and Closed-Loop Relabeling

The training procedure is also related to imitation learning under covariate shift. DAgger reduces compounding error by relabeling the states encountered by the learner rather than restricting supervision to expert trajectories [11]. We reuse that idea at the sensor level: a differentiable camera teacher generates relabeling targets for the states visited by the current policy, and the learned camera branch is then stabilized before flight adaptation. This separation is important because camera semantics and flight control are coupled but not identical objectives. The relabeling stage keeps the perception branch interpretable while the final flight-only optimization consumes the resulting observation distribution.

III. Method Overview

Let $\mathbf{x}_t$ be the quadrotor state, $\mathbf{c}_t = [P_t, E_t, G_t]^\top$ the active-depth camera state, and $\rho_t \in \{\text{glare}, \text{specular}, \text{dark}\}$ the local sensing regime. The proposed method learns a closed-loop map

$$\Pi_\theta : (D_t^{\text{obs}}, \mathbf{s}_t, \mathbf{c}_t, \mathbf{h}_t, \mathbf{q}_t) \mapsto (\mathbf{u}_t, \mathbf{c}_{t+1}^*, \mathbf{h}_{t+1}, \mathbf{q}_{t+1}), \quad (1)$$

where $\mathbf{u}_t$ is a local acceleration command and $\mathbf{c}_{t+1}^*$ is an absolute camera target. The main technical choice is to represent the sensor as a differentiable module $\mathcal{S}_\phi$ and to train the camera branch through teacher optimization and online relabeling, instead of relying on sparse collision gradients alone.

A. Closed-Loop Differentiable System

We denote by $\mathbf{x}_t$ the full simulator state at control step t , including vehicle pose, velocity, attitude, and collision margin. The symbol $\mathcal{E}$ denotes the environment description, including scene geometry, material attributes, light sources, and the randomized task parameters. The slit-navigation benchmark used in this paper is one instantiation of $\mathcal{E}$, but the formulation below only assumes an environment that can provide geometry and sensing-regime parameters. The active depth camera state is

$$\mathbf{c}_t = [P_t, E_t, G_t]^\top \in [0, 1]^3, \quad (2)$$

where P_t , E_t , and G_t are the normalized projector power, exposure, and gain. The variable ρ_t indexes the local sensing condition induced by $\mathcal{E}$, such as glare, specular reflection, or low-reflectance material.

The closed-loop rollout is written as

$$D_t^{\text{raw}} = \mathcal{R}(\mathbf{x}_t, \mathcal{E}), \quad (3)$$

$$(D_t^{\text{obs}}, Q_t, M_t) = \mathcal{S}_\phi(D_t^{\text{raw}}, \mathbf{c}_t, \rho_t, \mathbf{x}_t), \quad (4)$$

$$\mathbf{s}_t = \psi_{\text{flight}}(\mathbf{x}_t, \mathbf{x}_{\text{goal}}, \mathbf{c}_t), \quad (5)$$

$$\mathbf{m}_t = \psi_{\text{cam}}(\mathbf{x}_t, \mathbf{c}_t), \quad (6)$$

$$(\mathbf{u}_t, \mathbf{c}_{t+1}^*, \mathbf{h}_{t+1}, \mathbf{q}_{t+1}) = \Pi_\theta(D_t^{\text{obs}}, \mathbf{s}_t, \mathbf{c}_t, \mathbf{m}_t, \mathbf{h}_t, \mathbf{q}_t), \quad (7)$$

$$\mathbf{c}_{t+1} = \alpha \mathbf{c}_t + (1 - \alpha) \mathbf{c}_{t+1}^*, \quad \alpha = 0.7, \quad (8)$$

$$\mathbf{x}_{t+1} = f_{\text{dyn}}(\mathbf{x}_t, \mathbf{u}_t, \Delta t_t). \quad (9)$$

Here $\mathcal{R}$ is the native geometric ray renderer and $D_t^{\text{raw}} \in \mathbb{R}^{H \times W}$ is the ideal metric depth before sensor degradation. In our implementation, this raw geometry is generated by the CUDA routine `quadsim_cuda.render_depth`; the differentiable path needed by the active camera is provided by the subsequent sensor-response model. The active-depth sensor $\mathcal{S}_\phi$ maps this ideal depth and the camera state to the policy observation D_t^{obs} , a continuous quality map Q_t , and a soft validity map $M_t \in [0, 1]^{H \times W}$. The parameters ϕ collect the sensor-model constants, including range attenuation, exposure-time mapping, gain mapping, motion-blur proxy, noise proxy, and scene-effect masks.

The encoder ψ_{flight} builds the flight state $\mathbf{s}_t$ from goal-relative local velocity, body attitude, safety margin, and the current camera state when camera-state observation is enabled. In contrast, ψ_{cam} builds the camera-motion descriptor $\mathbf{m}_t$ from local velocity and attitude together with $\mathbf{c}_t$, but it does not use the goal vector. This design prevents the camera branch from learning a privileged schedule tied to the nominal slit geometry; camera commands must be conditioned on image quality, current registers, and motion. The policy Π_θ contains two recurrent states: $\mathbf{h}_t$ for flight control and $\mathbf{q}_t$ for camera control. The camera target $\mathbf{c}_{t+1}^*$ is smoothed by the first-order update (8); in the implementation $\alpha = 0.7$. The control interval satisfies

$$\Delta t_t = \Delta t_{\text{base}} + \kappa_\tau \tau(E_t), \quad (10)$$

where $\tau(E_t)$ is the effective exposure time and κ_τ is a small scaling factor. Thus, camera settings affect the closed

loop both through the observation model and through the timing of the rollout.

For a horizon T , the differentiable training objective can be written as

$$\mathcal{J}(\theta) = \sum_{t=0}^{T-1} \ell_{\text{nav}}(\mathbf{x}_t, \mathbf{u}_t) + \ell_{\text{sens}}(M_t, \mathbf{c}_t, \mathbf{x}_t). \quad (11)$$

The navigation loss ℓ_{nav} contains velocity tracking, smoothness, and collision terms, while ℓ_{sens} regularizes depth validity and camera operation. Because D_t^{obs} is generated by $\mathcal{S}_\phi$ rather than treated as an exogenous input, a downstream task loss can provide a camera gradient. A one-step dependency is

$$\begin{aligned} \frac{\partial \mathcal{J}}{\partial \mathbf{c}_t} &= \frac{\partial \mathcal{J}}{\partial D_t^{\text{obs}}} \frac{\partial D_t^{\text{obs}}}{\partial \mathbf{c}_t} + \frac{\partial \mathcal{J}}{\partial M_t} \frac{\partial M_t}{\partial \mathbf{c}_t} \\ &+ \frac{\partial \mathcal{J}}{\partial \mathbf{x}_{t+1}} \frac{\partial \mathbf{x}_{t+1}}{\partial \mathbf{u}_t} \frac{\partial \mathbf{u}_t}{\partial D_t^{\text{obs}}} \frac{\partial D_t^{\text{obs}}}{\partial \mathbf{c}_t} + \frac{\partial \mathcal{J}}{\partial \Delta t_t} \frac{\partial \Delta t_t}{\partial \mathbf{c}_t}. \end{aligned} \quad (12)$$

The first two terms describe direct sensing supervision through observed depth and validity. The third term describes task-level feedback that passes through the policy and dynamics. The final term captures the exposure-dependent timing effect. If $\partial D_t^{\text{obs}} / \partial \mathbf{c}_t$ and $\partial M_t / \partial \mathbf{c}_t$ are unavailable, these gradients collapse to a fixed-sensor formulation. Equation (12) is therefore the mathematical motivation for differentiable active perception.

B. Differentiable Active-Depth Model

The active-depth model follows the same modular principle as recent physics-based differentiable camera simulators: a geometric renderer first produces ideal scene measurements, and a differentiable camera model then converts them into sensor observations under explicit camera settings [5]. Our model is not a full photorealistic camera pipeline. It is a task-oriented active-stereo proxy that keeps the physical failure modes most relevant to fast slit navigation: range-dependent active return, ambient infrared washout, gain-dependent noise, motion blur, depth-edge matching failure, and material-dependent degradation.

An active stereo depth camera such as a D455-like sensor emits an infrared pattern, observes the reflected pattern with stereo imagers, and estimates depth by local correspondence. The returned depth therefore depends not only on geometry, but also on projector power, exposure, gain, surface reflectance, ambient infrared illumination, and vehicle motion. We write the raw geometric depth as $Z_t \equiv D_t^{\text{raw}}$ and decompose the model into a native geometric stage and a differentiable sensor stage.

Native geometric depth. Let $R_t^{\mathcal{E}} \in SO(3)$ and $\mathbf{p}_t^{\mathcal{E}}$ be the camera orientation and position in the environment frame after applying the fixed body-to-camera transform and the scene-frame transform. Denote the three columns of $R_t^{\mathcal{E}}$

by $(\mathbf{r}_{x,t}, \mathbf{r}_{y,t}, \mathbf{r}_{z,t})$. For pixel (i, j) , where $i = 0, \dots, H - 1$ and $j = 0, \dots, W - 1$, the CUDA renderer uses the ray

$$\eta_i = \left(2 \frac{i + 1/2}{H} - 1\right) f_y - \epsilon_r, \quad f_y = f_x \frac{H}{W}, \quad (13)$$

$$\xi_j = \left(2 \frac{j + 1/2}{W} - 1\right) f_x - \epsilon_r, \quad (14)$$

$$\mathbf{d}_{tij} = \mathbf{r}_{x,t} - \eta_i \mathbf{r}_{z,t} - \xi_j \mathbf{r}_{y,t}, \quad (15)$$

where $f_x = \tan(\text{FOV}_x/2)$ and ϵ_r is a small numerical offset. The ideal depth is the nearest positive ray intersection with the environment geometry:

$$Z_{tij} = \min_{\lambda > 0} \{ \lambda \mid \mathbf{p}_t^\mathcal{E} + \lambda \mathbf{d}_{tij} \in \mathcal{G}_\mathcal{E} \}. \quad (16)$$

The set $\mathcal{G}_\mathcal{E}$ contains the simulator primitives used by the benchmark, including voxels, cylindrical obstacles, spherical obstacles, walls, and other drones when present. Equation (16) describes the role of `quad-sim_cuda.render_depth`: it provides the metric geometry that an ideal pinhole depth sensor would see before active infrared failures are applied.

Differentiable active-stereo response. The sensor stage first clamps the native depth into the valid measurement range,

$$\bar{Z}_{tij} = \text{clip}(Z_{tij}, z_{\min}, z_{\max}), \quad (17)$$

and computes a local depth-discontinuity score from the 3×3 pixel neighborhood $\mathcal{N}_{ij}$:

$$Z_{tij}^+ = \max_{(m,n) \in \mathcal{N}_{ij}} \bar{Z}_{tmn}, \quad (18)$$

$$Z_{tij}^- = \min_{(m,n) \in \mathcal{N}_{ij}} \bar{Z}_{tmn}, \quad (19)$$

$$\delta_{tij} = \text{clip} \left(\frac{Z_{tij}^+ - Z_{tij}^-}{\bar{Z}_{tij} + \epsilon_z}, 0, 1 \right). \quad (20)$$

The term δ_{tij} approximates the correspondence difficulty near occlusion boundaries and narrow-aperture edges. The environment also projects a continuous material or illumination mask

$$\Omega_{tij} = m_{\rho_t}(\bar{Z}_{tij}, \mathbf{x}_t, \mathcal{E}) \in [0, 1], \quad (21)$$

where $\Omega_{tij} = 0$ denotes ordinary material and larger values denote a localized sensing hazard. In the implementation, side-wall masks are computed from the ray hit location in the scene frame, while glare masks are localized around the aperture and its halo. The pose used in this mask projection is detached in the sensor branch; this keeps the active-sensing gradient focused on the camera registers rather than on discontinuous ray-intersection topology.

The normalized camera command is $\mathbf{c}_t = [P_t, E_t, G_t]^\top$. We write $p_t = \text{clip}(P_t, 0, 1)$, $e_t = \text{clip}(E_t, 0, 1)$, and $g_t = \text{clip}(G_t, 0, 1)$. The exposure time and ISO-like gain are

$$\tau(E_t) = t_{\min} + t_{\text{span}} e_t, \quad (22)$$

$$\Gamma(G_t) = \Gamma_0 + \Gamma_s \frac{(g_t + \epsilon_g)^\gamma - \epsilon_g^\gamma}{(1 + \epsilon_g)^\gamma - \epsilon_g^\gamma}. \quad (23)$$

Here τ controls how long the infrared images accumulate photons and Γ controls the amplification of the received

signal. The active and passive infrared returns are modeled as

$$S_{tij}^{\text{act}} = \frac{k_a p_t \tau(E_t)}{\bar{Z}_{tij}^2 + c_d}, \quad (24)$$

$$S_t^{\text{pass}} = k_p \tau(E_t) \sqrt{\Gamma(G_t)}, \quad (25)$$

$$S_{tij} = S_{tij}^{\text{act}} + S_t^{\text{pass}}. \quad (26)$$

The inverse-square term in (24) captures range attenuation of the emitted pattern. The passive term captures ambient or texture-assisted stereo cues that scale with exposure and gain.

The base quality score combines signal-to-noise ratio, ambient washout, motion, edge difficulty, and range saturation:

$$I_{tij}^{\text{amb}} = a_0 + a_1 \Omega_{tij}, \quad (27)$$

$$B_t^{\text{mot}} = \text{clip}(k_m \|\mathbf{v}_t\|_2 \tau(E_t), 0, b_{\max}), \quad (28)$$

$$W_{tij}^{\text{out}} = \frac{I_{tij}^{\text{amb}} \tau(E_t)}{S_{tij}^{\text{act}} + c_w}, \quad (29)$$

$$N_{tij}^{\text{shot}} = \frac{n_0(n_1 + n_2 \Gamma(G_t))}{S_{tij} + c_n}, \quad (30)$$

$$\chi_{tij} = \frac{S_{tij}}{b_0 + b_1 I_{tij}^{\text{amb}} + b_2 N_{tij}^{\text{shot}} + b_3 B_t^{\text{mot}} (b_4 + \delta_{tij})}, \quad (31)$$

$$R_{tij} = [\bar{Z}_{tij}/z_{\max} - r_0]_+, \quad (32)$$

$$Q_{tij}^0 = \sigma(\beta_s \chi_{tij} - \beta_w W_{tij}^{\text{out}} - \beta_e \delta_{tij} - \beta_r R_{tij}). \quad (33)$$

The variables I^{amb} , W^{out} , N^{shot} , and χ denote ambient infrared level, washout, noise proxy, and signal-to-noise proxy, respectively. All coefficients in (24)–(33) are contained in ϕ and are chosen to reproduce D455-like trends rather than device firmware exactly.

Localized physical regimes are then applied as differentiable penalties and bonuses:

$$C_{\rho,t} = B_{\rho_t}(p_t, e_t, g_t) - A_{\rho_t}(p_t, e_t, g_t), \quad (34)$$

$$Q_{tij} = \text{clip}(Q_{tij}^0 + \Omega_{tij} C_{\rho,t}, 0, 1). \quad (35)$$

For glare, A_ρ increases with exposure saturation, gain saturation, and insufficient projector power in the aperture halo, while B_ρ rewards the high-power, low-exposure, low-gain rescue region. For specular side walls, A_ρ increases with active-return blooming caused by high power, exposure, or gain, while B_ρ rewards conservative register settings that avoid multipath-like loss. For dark material, A_ρ is high unless exposure, gain, and projector power jointly recover the weak return; B_ρ adds a bounded rescue bonus, but the noise and motion penalties in (33) remain active.

The continuous validity map and the observed depth are

$$M_{tij} = \sigma \left(\frac{Q_{tij} - q_0}{\sigma_q} \right), \quad (36)$$

$$V_{tij} = \mathbb{I}[M_{tij} > 1/2], \quad (37)$$

$$\widetilde{M}_{tij} = \text{sg}(V_{tij} - M_{tij}) + M_{tij}, \quad (38)$$

$$D_{tij}^{\text{obs}} = \bar{Z}_{tij} \widetilde{M}_{tij}, \quad Q_{tij}^{\text{obs}} = Q_{tij} \widetilde{M}_{tij}. \quad (39)$$

The operator $\text{sg}(\cdot)$ stops gradients. Hence (39) behaves as a hard validity mask in the forward pass, but as the differentiable probability M_{tij} in the backward pass. This is critical for active perception: invalid pixels must be physically absent from the depth image seen by the policy, while their probability of becoming valid must still provide a gradient to P_t , E_t , and G_t .

For any downstream loss $\mathcal{L}$, define the scalar sensitivity passed to the quality map as

$$\zeta_{tij} = \frac{\partial \mathcal{L}}{\partial Q_{tij}} + \left(\frac{\partial \mathcal{L}}{\partial M_{tij}} + \bar{Z}_{tij} \frac{\partial \mathcal{L}}{\partial D_{tij}^{\text{obs}}} \right) \frac{M_{tij}(1 - M_{tij})}{\sigma_q}. \quad (40)$$

The sensor backward pass then aggregates pixelwise camera-register gradients:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{c}_t} = \sum_{i,j} \zeta_{tij} \frac{\partial Q_{tij}}{\partial \mathbf{c}_t}. \quad (41)$$

The fused CUDA implementation computes (41) over all pixels and returns gradients for (P_t, E_t, G_t) . The raw depth $\bar{Z}_t$ and the projected mask Ω_t are treated as constants in this branch, while $\partial Q_{tij}/\partial \mathbf{c}_t$ is evaluated with bounded local differences that exactly match the piecewise CUDA forward model. This design keeps the expensive ray-intersection stage native and fast, but makes the active-depth image formation process differentiable with respect to the physical camera registers that the policy controls.

C. Split-Stem Policy

The policy network is implemented as a split-stem recurrent architecture, as shown in Fig. 2. The design separates two roles that are coupled in the rollout but should not share all feature semantics during training. The flight branch must extract geometry useful for acceleration control, whereas the camera branch must preserve image-quality cues needed to choose projector power, exposure, and gain. A single shared visual encoder would allow later flight-only adaptation to move the representation used by the camera controller, corrupting the semantics learned from the differentiable camera teacher. We therefore use one visual stem for flight, an independently parameterized copy for camera control, and a small camera-specific spatial adapter.

The observed depth is first mapped to a compact two-channel representation before entering either recurrent branch. Since invalid pixels are represented by zero-valued observations after the active-depth sensor model, define

$$V_{tij}^d = \mathbb{I}[D_{tij}^{\text{obs}} \geq z_{\min}], \quad (42)$$

$$\bar{D}_{tij} = \begin{cases} \text{clip}(D_{tij}^{\text{obs}}, z_{\min}, z_{\max}), & V_{tij}^d = 1, \\ z_{\max}, & V_{tij}^d = 0. \end{cases} \quad (43)$$

Two channels are then constructed:

$$\phi_t^{\text{near}} = \text{Pool}_{\max}^{H_d \times W_d} \left[V_t^d \odot \text{clip} \left(\frac{1/\bar{D}_t - 1/z_{\max}}{1/z_{\min} - 1/z_{\max}}, 0, 1 \right) \right], \quad (44)$$

$$\phi_t^{\text{far}} = \text{Pool}_{\text{avg}}^{H_d \times W_d} \left[V_t^d \odot \text{clip} \left(\frac{\bar{D}_t - z_{\min}}{z_{\max} - z_{\min}}, 0, 1 \right) \right], \quad (45)$$

$$\Phi_t = 2[\phi_t^{\text{near}}, \phi_t^{\text{far}}] - 1. \quad (46)$$

The near channel emphasizes collision-relevant foreground geometry by inverse depth and adaptive max pooling. The far channel emphasizes open space and aperture cues by metric range and adaptive average pooling. In the experiments $H_d = 12$ and $W_d = 16$.

The flight visual stem F_f consists of three convolutional blocks with channels 32, 64, and 128, a stride-2 downsampling in the second block, adaptive average pooling to 3×6 , flattening, and a linear projection to 192 dimensions. Let

$$\mathbf{z}_t^f = F_f(\Phi_t) \in \mathbb{R}^{192}, \quad (47)$$

$$\bar{\mathbf{z}}_t^f = \text{LN}(\mathbf{z}_t^f), \quad \bar{\mathbf{s}}_t = \text{LN}(W_s \mathbf{s}_t), \quad (48)$$

$$\mathbf{g}_t = \sigma \left(W_{g2} \varphi \left(W_{g1} [\bar{\mathbf{z}}_t^f, \bar{\mathbf{s}}_t] \right) \right), \quad (49)$$

$$\mathbf{r}_t = \varphi \left(\mathbf{g}_t \odot \bar{\mathbf{z}}_t^f + (1 - \mathbf{g}_t) \odot \bar{\mathbf{s}}_t \right). \quad (50)$$

Here LN denotes layer normalization and φ is LeakyReLU with slope 0.05. The gate $\mathbf{g}_t$ is a learned elementwise mixture between visual geometry and the flight state. The recurrent flight state and action head are

$$\tilde{\mathbf{h}}_{t+1} = \text{GRU}_f(\mathbf{r}_t, \mathbf{h}_t), \quad (51)$$

$$\mathbf{h}_{t+1} = \text{LN} \left(\tilde{\mathbf{h}}_{t+1} + 0.1 R_h(\tilde{\mathbf{h}}_{t+1}) \right), \quad (52)$$

$$\mathbf{u}_t = W_u \varphi(\mathbf{h}_{t+1}), \quad (53)$$

where R_h is a small residual MLP initialized near zero.

The camera branch uses an independent visual stem F_c with the same convolutional layout as F_f . It is initialized from F_f for representation compatibility, but the two stems do not share parameters after initialization. A lightweight spatial adapter G_c extracts low-capacity local quality cues with 4-channel convolutions and 2×3 pooling. The camera image feature is

$$\mathbf{z}_t^{c,0} = F_c(\Phi_t), \quad (54)$$

$$\mathbf{z}_t^{c,s} = W_{sp} G_c(\Phi_t), \quad (55)$$

$$\mathbf{z}_t^c = \text{LN} \left(\mathbf{z}_t^{c,0} + 0.25 A_c(\mathbf{z}_t^{c,0}) + 0.35 \mathbf{z}_t^{c,s} \right), \quad (56)$$

where A_c is the residual camera image adapter. The camera input excludes the goal vector. It uses only the image feature, the normalized current camera state $\hat{\mathbf{c}}_t = 2\mathbf{c}_t - 1$, and a motion descriptor

$$\mathbf{m}_t = [\text{clip}(\mathbf{v}_t^{\text{local}}/v_{\max}, -2, 2), \mathbf{R}_t[:, 3]] \in \mathbb{R}^6, \quad (57)$$

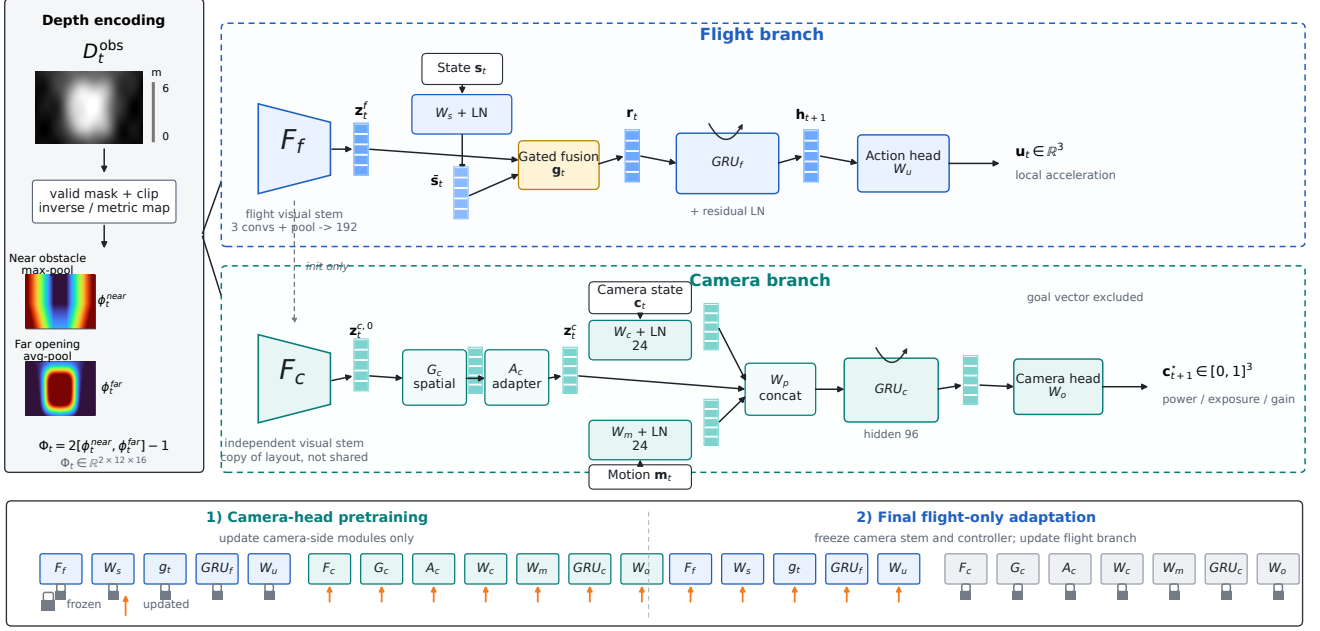


Fig. 2. The proposed split-stem recurrent policy architecture. The observed metric depth is converted into near-obstacle and far-opening channels. The flight branch fuses visual geometry with the flight state and outputs a local acceleration command. The camera branch uses an independent visual stem, a lightweight spatial adapter, the normalized camera state, and the local motion descriptor to output the next absolute camera target. Camera-head pretraining optimizes only camera-side parameters, while final flight-only adaptation freezes the camera stem and controller.

where $\mathbf{R}_t[:, 3]$ is the body up direction. The camera recurrent controller is

$$\mathbf{e}_t^c = \text{LN}(W_c \hat{\mathbf{c}}_t), \quad \mathbf{e}_t^m = \text{LN}(W_m \mathbf{m}_t), \quad (58)$$

$$\mathbf{a}_t^c = \varphi(W_p[\mathbf{z}_t^c, \mathbf{e}_t^c, \mathbf{e}_t^m]), \quad (59)$$

$$\mathbf{q}_{t+1} = \text{LN}(\text{GRU}_c(\mathbf{a}_t^c, \mathbf{q}_t)), \quad (60)$$

$$\mathbf{c}_{t+1}^* = \sigma(W_o \varphi(\mathbf{q}_{t+1})). \quad (61)$$

The dimensions are 24 for the camera-state embedding, 24 for the motion-state embedding, and 96 for the camera GRU. The output $\mathbf{c}_{t+1}^* \in [0, 1]^3$ is an absolute target for projector power, exposure, and gain; the temporal smoothing in (8) is applied outside the network.

The same architectural partition is enforced during optimization. During camera-head pretraining, all non-camera parameters are held fixed and the flight recurrent state is treated as a detached context variable. During the final flight-only stage, all camera-side modules remain active in the forward pass but are excluded from parameter updates:

$$\Theta_{\text{cam}} = \{F_c, G_c, W_{sp}, A_c, W_c, W_m, W_p, \text{GRU}_c, W_o, \text{camera normalization layers}\}. \quad (62)$$

D. Teacher Optimization

The camera teacher is a state-wise differentiable perception oracle. It does not prescribe the flight action. Instead, at a visited simulator state $\xi_t = (\mathbf{x}_t, \mathbf{c}_t, \rho_t)$, it asks which next camera register would make the local depth observation reliable while remaining physically economical and temporally smooth. The vehicle pose,

velocity, scene, and raw geometry are fixed during this local solve; only the next camera target is optimized. Let $\bar{\mathbf{c}} = [\bar{P}, \bar{E}, \bar{G}]^\top = \sigma(\boldsymbol{\eta}) \in [0, 1]^3$ be a candidate target parameterized by unconstrained logits $\boldsymbol{\eta}$. Re-rendering the active-depth response at the same state gives

$$(D^{\text{obs}}(\bar{\mathbf{c}}), Q(\bar{\mathbf{c}}), M(\bar{\mathbf{c}})) = \mathcal{S}_\phi(D_t^{\text{raw}}, \bar{\mathbf{c}}, \rho_t, \mathbf{x}_t). \quad (63)$$

Because $\mathcal{S}_\phi$ is differentiable with respect to $\bar{\mathbf{c}}$, losses on depth validity and sensor quality can be backpropagated directly to projector power, exposure, and gain.

The primary reliability statistic is a lower-tail patch fill rate. Let $R_p = 6$ and $C_p = 8$ be the pooling grid, and let $\beta \in (0, 1]$ be the lower-tail fraction. We pool the soft validity map $M(\bar{\mathbf{c}})$ into patch occupancies and average the K least reliable patches:

$$\mathbf{p}(\bar{\mathbf{c}}) = \text{vec}\left(\text{AvgPool}_{R_p \times C_p}(M(\bar{\mathbf{c}}))\right), \quad (64)$$

$$K = \lceil \beta R_p C_p \rceil, \quad (65)$$

$$r_{\text{fill}}(\bar{\mathbf{c}}) = \frac{1}{K} \sum_{i \in \mathcal{B}_K(\mathbf{p})} p_i, \quad (66)$$

where $\mathcal{B}_K(\mathbf{p})$ indexes the K smallest elements of $\mathbf{p}$. This conditional-value-at-risk style statistic penalizes local holes in the depth map rather than allowing high-confidence background pixels to hide a failed aperture region.

The teacher objective is

$$\begin{aligned} \bar{\mathbf{c}}_{t+1} &= \arg \min_{\bar{\mathbf{c}} \in [0,1]^3} \mathcal{L}_{\text{teach}}(\bar{\mathbf{c}}), \\ \mathcal{L}_{\text{teach}} &= \lambda_f [r_{\min} - r_{\text{fill}}(\bar{\mathbf{c}})]_+^2 + \lambda_s \|\bar{\mathbf{c}} - \mathbf{c}_t\|_2^2 \\ &\quad + \lambda_P [\bar{P} - P_0]_+^2 + \lambda_b \|\mathbf{v}_t\|_2^2 \tau(\bar{E})^2 \\ &\quad + \lambda_G \bar{G}^2 + \lambda_0 \omega(\bar{\mathbf{c}}) \|\bar{\mathbf{c}} - \mathbf{c}_{\text{nom}}\|_2^2. \end{aligned} \quad (67)$$

Here $r_{\min}$ is the required fill level, P_0 is the nominal power baseline, $\tau(\bar{E})$ maps normalized exposure to physical integration time, and $\mathbf{c}_{\text{nom}}$ is the nominal camera setting. The terms have distinct physical roles. The fill penalty raises depth support in the least reliable image regions. The smoothness term avoids frame-to-frame camera jumps. The power term discourages unnecessary projector intensity above the baseline. The blur term penalizes long exposure during fast motion, and the gain term discourages amplified noise. The final recovery term returns the camera toward the nominal setting only after the depth map is already healthy:

$$\omega(\bar{\mathbf{c}}) = \text{sg} \left[\text{clip} \left(\frac{r_{\text{fill}}(\bar{\mathbf{c}}) - r_{\min}}{m_{\text{fill}}}, 0, 1 \right) \right], \quad (68)$$

where m_{fill} is a recovery margin and $\text{sg}[\cdot]$ denotes a stop-gradient operation. Thus, nominal recovery does not weaken the gradient that first restores depth validity.

The essential point is that the teacher label is produced by differentiating through the active-depth image formation model:

$$\nabla_{\boldsymbol{\eta}} \mathcal{L}_{\text{teach}} = \left(\frac{\partial \mathcal{L}_{\text{teach}}}{\partial M} \frac{\partial M}{\partial \bar{\mathbf{c}}} + \frac{\partial \mathcal{L}_{\text{teach}}}{\partial Q} \frac{\partial Q}{\partial \bar{\mathbf{c}}} \right) \frac{\partial \bar{\mathbf{c}}}{\partial \boldsymbol{\eta}} + \nabla_{\boldsymbol{\eta}} \mathcal{L}_{\text{reg}}. \quad (69)$$

Here $\mathcal{L}_{\text{reg}}$ denotes the smoothness, power, blur, gain, and nominal terms in (67). Without differentiable perception, the factors $\partial M / \partial \bar{\mathbf{c}}$ and $\partial Q / \partial \bar{\mathbf{c}}$ would be unavailable, and the camera label would have to be designed by hand.

The logits are initialized from the current camera state and optimized by Adam:

$$\begin{aligned} \boldsymbol{\eta}^0 &= \text{logit}(\text{clip}(\mathbf{c}_t, \epsilon, 1 - \epsilon)), \\ \boldsymbol{\eta}^{k+1} &= \text{Adam}(\boldsymbol{\eta}^k, \nabla_{\boldsymbol{\eta}^k} \mathcal{L}_{\text{teach}}(\sigma(\boldsymbol{\eta}^k))), \\ \bar{\mathbf{c}}_{t+1} &= \sigma(\boldsymbol{\eta}^{K_{\text{teach}}}). \end{aligned} \quad (70)$$

For the reported DAgger relabeling, we use $K_{\text{teach}} = 120$ Adam steps with learning rate 0.08. During this relabeling, teacher-camera EMA is disabled; the learned camera, not the teacher, determines the next observation distribution.

E. Camera Supervision and DAgger Relabeling

The teacher collection yields a sequence dataset

$$\mathcal{D} = \{(D_{b,t}^{\text{obs}}, \mathbf{s}_{b,t}, \mathbf{c}_{b,t}, \mathbf{m}_{b,t}, \bar{\mathbf{c}}_{b,t+1}, \rho_b, x_{b,t}^{\text{loc}}, r_{\text{fill},b,t})\}_{b,t}. \quad (71)$$

The camera branch is trained as a recurrent regressor from the policy observations and camera-side state to

Algorithm 1 State-Wise Teacher Collection and Camera Pretraining

Require: frozen flight policy Π_{θ_0} , scenarios $\mathcal{R}$, rollouts N , horizon T

Ensure: camera-pretrained policy θ_{pre}

- 1: $\mathcal{D}_0 \leftarrow \emptyset$
- 2: for $\rho \in \mathcal{R}$ do
- 3: for $n = 1, \dots, N$ do
- 4: reset the simulator under sensing regime ρ
- 5: initialize $\mathbf{c}_0 = \mathbf{c}_{\text{nom}}$
- 6: for $t = 0, \dots, T - 1$ do
- 7: render D_t^{obs} by (3)–(4)
- 8: build $\mathbf{s}_t$ and $\mathbf{m}_t$ by (5)–(6)
- 9: infer flight command $(\mathbf{u}_t, \mathbf{c}_{t+1}^*) \leftarrow \Pi_{\theta_0}(D_t^{\text{obs}}, \mathbf{s}_t, \mathbf{c}_t, \mathbf{m}_t)$
- 10: solve $\bar{\mathbf{c}}_{t+1}$ by (67)–(70)
- 11: append $(D_t^{\text{obs}}, \mathbf{s}_t, \mathbf{c}_t, \mathbf{m}_t, \bar{\mathbf{c}}_{t+1}, \rho, x_t^{\text{loc}}, r_{\text{fill}})$ to $\mathcal{D}_0$
- 12: update the collection camera by the selected rollout schedule and step dynamics with $\mathbf{u}_t$
- 13: end for
- 14: end for
- 15: end for
- 16: optimize only Θ_{cam} on $\mathcal{D}_0$ using (72)
- 17: return θ_{pre}

the differentiable teacher target. With the hidden states unrolled over each sequence, the supervised objective is

$$\begin{aligned} \mathcal{L}_{\text{sup}} &= \frac{1}{BT} \sum_{b=1}^B \sum_{t=0}^{T-1} \rho_{\delta}(\mathbf{c}_{b,t+1}^* - \bar{\mathbf{c}}_{b,t+1}) \\ &\quad + \lambda_{\Delta c} \frac{1}{B(T-1)} \sum_{b=1}^B \sum_{t=1}^{T-1} \|\mathbf{c}_{b,t+1}^* - \mathbf{c}_{b,t}^*\|_2^2, \end{aligned} \quad (72)$$

where ρ_{δ} is the SmoothL1 penalty applied component-wise to projector power, exposure, and gain. During this stage, $\Theta \setminus \Theta_{\text{cam}}$ is fixed, and the flight recurrent state is treated as context rather than as a trainable path. The optimization therefore transfers the differentiable teacher’s sensor semantics into the camera controller without disturbing the flight controller.

Offline camera pretraining alone is not sufficient, because the camera head changes the future observations that it will later receive. Let $\mu(\theta_{\text{cam}})$ denote the rollout distribution induced by a learned camera controller. A camera trained only on $\mathcal{D}_0$ is queried under $\mu(\theta_{\text{cam}})$ at deployment, which generally differs from the teacher-smoothed or fixed-camera collection distribution. We correct this covariate shift with DAgger-style online relabeling at the sensor level:

$$\mathcal{D}_{\text{dag}} = \left\{ (\xi_t, \bar{\mathbf{c}}_{t+1}) \mid \xi_t \sim \mu(\theta_{\text{pre}}), \bar{\mathbf{c}}_{t+1} = \arg \min_{\bar{\mathbf{c}}} \mathcal{L}_{\text{teach}} \right\}. \quad (73)$$

In this rollout, the student camera output is applied through (8) and fully determines the next depth observation. The teacher is queried on the states actually visited by the student-induced camera distribution, so the relabels

Algorithm 2 DAgger Relabeling and Flight-Only Adaptation

Require: camera-pretrained policy θ_{pre} , teacher objective (67)

Ensure: final split-stem policy θ^*

```

  DAgger relabeling
  1:  $\mathcal{D}_{\text{dag}} \leftarrow \emptyset$ 
  2: for each learned-camera rollout do
  3:   for  $t = 0, \dots, T - 1$  do
  4:     render the current observation using the student camera state
  5:     infer  $(\mathbf{u}_t, \mathbf{c}_{t+1}^*)$  with  $\Pi_{\theta_{\text{pre}}}$ 
  6:     label the visited state:  $\bar{\mathbf{c}}_{t+1} \leftarrow \arg \min_{\bar{\mathbf{c}}} \mathcal{L}_{\text{teach}}$ 
  7:     append the labeled sample to  $\mathcal{D}_{\text{dag}}$ 
  8:     update  $\mathbf{c}_{t+1}$  by (8) and step dynamics with  $\mathbf{u}_t$ 
  9:   end for
  10: end for
  11: retrain  $\Theta_{\text{cam}}$  on  $\mathcal{D}_{\text{dag}}$  using (72)
  Flight-only adaptation
  12: freeze  $\Theta_{\text{cam}}$  and detach the camera recurrent path
  13: optimize  $\Theta_f = \Theta \setminus \Theta_{\text{cam}}$  by (75) under learned-camera rollouts
  14: return  $\theta^*$ 
  
```

target the distribution that the learned camera will see at deployment. The final relabeled dataset contains 144 sequences and 11,520 state-wise labels. Its teacher mean/std for $P/E/G$ are 0.584/0.408/0.403 and 0.164/0.197/0.181, respectively, indicating that the teacher produces nontrivial, scene-dependent active-sensing commands.

F. Flight Adaptation Objective

After DAgger, the camera branch defines a stable active-sensing policy. The final stage then adapts only the flight branch to the observation distribution created by that learned camera. This design is deliberate: gradients from the navigation loss are allowed to improve flight control, but they are not allowed to overwrite the camera semantics learned from the differentiable perception teacher. Formally, let $\Theta_f = \Theta \setminus \Theta_{\text{cam}}$. During flight-only adaptation,

$$\mathbf{c}_{t+1}^* = \text{sg}[\Pi_{\Theta_{\text{cam}}}^c(D_t^{\text{obs}}, \mathbf{c}_t, \mathbf{m}_t, \mathbf{q}_t)], \quad \theta_{\text{cam}} \text{ fixed}, \quad (74)$$

where $\Pi_{\Theta_{\text{cam}}}^c$ denotes the camera branch of Π_{θ} . The camera still affects the forward rollout through (4) and (8), but its parameters and recurrent state do not receive gradients.

The flight loss is

$$\begin{aligned} \mathcal{L}_{\text{flight}}(t) &= \lambda_v \mathcal{L}_v(t) + \lambda_a \mathcal{L}_{\text{acc}}(t) \\ &\quad + \lambda_j \mathcal{L}_{\text{jerk}}(t) + \lambda_c \mathcal{L}_{\text{collide}}(t), \\ \theta^* &= \arg \min_{\Theta_f} \mathbb{E}_{\rho, \mathbf{x}_0} \sum_{t=0}^{T-1} \mathcal{L}_{\text{flight}}(t). \end{aligned} \quad (75)$$

The camera parameter set Θ_{cam} is frozen throughout this optimization. The velocity term tracks a local goal-directed reference velocity over a short temporal window,

$$\mathcal{L}_v(t) = \rho_{\delta} \left(\left\| \frac{1}{W} \sum_{i=0}^{W-1} \mathbf{v}_{t-i} - \mathbf{v}_t^{\text{ref}} \right\|_2 \right), \quad (76)$$

while the control-effort and smoothness terms are

$$\mathcal{L}_{\text{acc}}(t) = \|\mathbf{u}_t\|_2^2, \quad \mathcal{L}_{\text{jerk}}(t) = \left\| \frac{\mathbf{u}_t - \mathbf{u}_{t-1}}{\Delta t_t} \right\|_2^2. \quad (77)$$

Let $d_{t,k}$ be the signed clearance, after subtracting the vehicle safety margin, for the k -th short-horizon obstacle query. Negative values indicate penetration. The collision barrier is

$$\mathcal{L}_{\text{collide}}(t) = \frac{1}{K_o} \sum_{k=1}^{K_o} \zeta_{t,k} \text{softplus}(-\gamma d_{t,k}), \quad (78)$$

where γ controls the sharpness and $\zeta_{t,k}$ weights samples that are moving toward the obstacle more heavily. Camera regularizers are disabled in this stage. The resulting optimization is therefore a clean decomposition: differentiable perception produces and stabilizes the active camera policy, and differentiable dynamics adapts the flight policy to exploit the depth stream created by that camera.

IV. Experimental Setup

A. Evaluation Rationale

The experiments are organized around the central differentiability claim. If the proposed sensor model really closes the perception-simulation-control loop, then three outcomes should appear together. First, active camera control should improve closed-loop navigation compared with fixed or task-agnostic camera settings. Second, the improvement should be accompanied by healthier depth observations, because the mechanism is improved sensing rather than arbitrary flight regularization. Third, the camera response should be scene-specific near the slit: glare should suppress exposure and gain, whereas low-reflectance material should increase them.

This rationale determines the organization of the evaluation. Overall success and collision quantify the task-level endpoint of the gradient chain. Depth fill measures whether the observation stream remains usable. Phase-wise camera statistics and matched-pose depth sequences test whether the learned behavior is consistent with the intended active-sensing mechanism rather than a trajectory artifact.

B. Benchmark Configuration

Each episode lasts 60 control steps at 15 Hz, so the closed-loop horizon spans four seconds. The raw depth renderer produces a 96×72 image, and the policy receives a 48×36 encoded depth input. The task uses the same wall, slit, start, and goal geometry across all sensing regimes. The slit center is sampled laterally, and the entire scene can be randomly rotated in yaw. This rotation means that success requires using the local depth and state cues rather than a fixed world-frame direction.

The camera action is three-dimensional and normalized to $[0, 1]^3$. The fixed baseline uses the nominal setting. The random-fixed baseline samples a static camera setting for each episode and keeps it fixed. Learned-camera methods update the camera state through the smoothed camera command described above.

Setting	Value
Control frequency	15 Hz
Episode horizon	60 steps
Rendered depth resolution	96×72
Policy depth resolution	48×36
Camera action	power / exposure / gain in $[0, 1]^3$
Scenes	glare, dark, specular
Slit center	randomized laterally
Scene yaw	randomized
Teacher rollouts per scene	4
Teacher batch size	12
Teacher optimization steps	120
Camera pretraining epochs	180, then 120 after relabeling
Evaluation episodes	900 per primary method

TABLE I

Core simulation, training, and evaluation settings. Evaluation uses 300 episodes per scene for each primary method.

C. Compared Policies

The primary comparison isolates different ways of handling camera parameters. The proposed method uses the full pipeline: differentiable camera relabeling, DAGger-style online relabeling, split-stem camera freezing, and flight-only adaptation. Fixed camera tests whether a single nominal setting is sufficient. Random fixed camera tests whether merely exposing the flight policy to diverse static camera settings explains the result. The non-differentiable learned camera baseline tests whether a learned camera controller without the differentiable active-sensing training signal develops the same scene-specific behavior. The blind baseline removes depth information and provides a lower bound on geometry-free flight.

D. Metrics and Statistical Reporting

An episode is successful when the vehicle reaches the goal region without collision. Collision is recorded if the trajectory intersects the wall or violates the clearance proxy. We also report terminal distance to the goal and the mean valid-depth fill rate. Let $\mathbf{x}_t$ denote the drone position, F_t the fraction of valid depth pixels at time t , and ρ_t the minimum clearance to any obstacle. We summarize

$$\text{success} = \mathbb{I}[\|\mathbf{x}_T - \mathbf{x}_{\text{goal}}\|_2 \leq \delta \wedge \neg \text{collision}], \quad (79)$$

$$\text{collision} = \mathbb{I}[\exists t : \rho_t \leq \epsilon], \quad (80)$$

$$\text{fill} = \frac{1}{T} \sum_{t=1}^T F_t, \quad (81)$$

$$d_{\text{final}} = \|\mathbf{x}_T - \mathbf{x}_{\text{goal}}\|_2. \quad (82)$$

Binary outcomes are reported with Wilson 95% confidence intervals. Continuous episode metrics, including fill and terminal distance, are reported with bootstrap 95% confidence intervals.

Camera behavior is summarized within three local phases relative to the wall, where x_t denotes the scalar

local-forward component of $\mathbf{x}_t$ after transforming the position into the wall frame:

$$x_t \in \begin{cases} \text{before,} & x_t < -0.25 \text{ m,} \\ \text{near,} & |x_t| \leq 0.25 \text{ m,} \\ \text{after,} & x_t > 0.25 \text{ m.} \end{cases} \quad (83)$$

The near phase is the most important because the depth image then determines whether the vehicle can align with the slit and avoid the wall. We report episode-aggregated near-slit power, exposure, and gain, as well as the glare-dark camera separation measured by the mean absolute parameter difference. These phase-wise statistics are diagnostic rather than training targets.

V. Results

The results are presented as evidence for a closed-loop sensing mechanism. We first check that training converges, then test whether differentiable active sensing improves navigation, and finally inspect whether the learned sensor actions are physically interpretable near the aperture.

A. Training Curves Are a Convergence Check, Not the Main Evidence

Figure 3 reports training loss, success, and collision curves for the comparison methods. Their role is diagnostic rather than decisive: they show that the policies reached stable regimes and that the final flight-only run is not an incomplete optimization artifact. Final claims use held-out closed-loop evaluation with the same episode budget and scene distribution for every method.

For the final split-stem run, the training summary and held-out evaluation are consistent. The training summary reports approximately 0.750 success and 0.250 collision, while held-out evaluation gives 0.751 success and 0.249 collision. This agreement is notable because an earlier shared-stem variant produced a larger mismatch: the camera branch appeared useful during training, but its semantics were not stable under evaluation. The split-stem layout removes that failure mode by fixing the camera features during flight-only adaptation. The training curve therefore acts as a consistency check for the final system, not as a substitute for held-out evaluation.

B. Active Camera Control Improves Closed-Loop Navigation

The primary evaluation is summarized in Table III and Figure 4. The proposed policy succeeds in 0.751 of episodes, compared with 0.667 for fixed camera, 0.627 for random fixed camera, 0.580 for the non-differentiable learned-camera baseline, and 0.032 for the blind controller. The collision rate falls from 0.333 for fixed camera to 0.249 for the proposed policy. The improvement is not only a geometric-control effect: depth fill rises from 0.721 under fixed camera to 0.975 under the active policy. In Table III, bracketed values denote 95% confidence intervals. Success and collision are episode-level rates with

Method	Role	Description
Ours	main model	split-stem learned camera with flight-only adaptation
Fixed	baseline	nominal constant camera setting
Random fixed	baseline	episode-constant random camera setting
Non-diff.	baseline	learned camera without differentiable sensor training
Blind	baseline	zero-depth controller
Pretrain	diagnostic	camera-pretrained policy before flight adaptation
Dagger	diagnostic	relabelled camera policy before flight adaptation

TABLE II

Policies used in the quantitative comparison and diagnostic analysis. Only the first five methods are primary navigation baselines; pretrain and Dagger are included to separate camera semantics from flight adaptation.

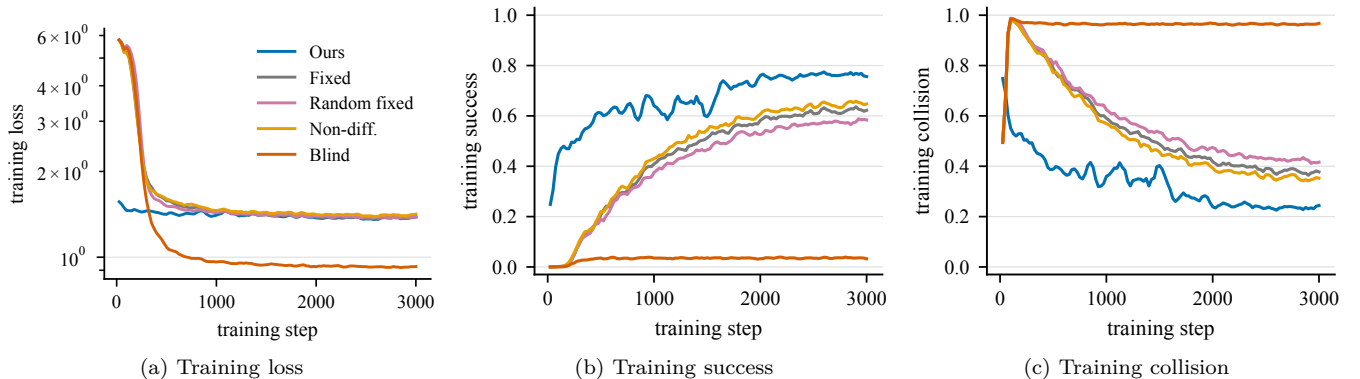


Fig. 3. Training diagnostics for the comparison policies. The curves document optimization behavior and convergence, whereas final claims use the held-out closed-loop evaluation in Table III and Figure 4.

Method	Success rate (95% CI)	Δ success vs. Fixed	Collision rate (95% CI)	Depth fill rate (95% CI)	Δ fill vs. Fixed
Ours	0.751 [0.722,0.778]	+0.084	0.249 [0.222,0.278]	0.975 [0.974,0.977]	+0.255
Fixed	0.667 [0.635,0.697]	+0.000	0.333 [0.303,0.365]	0.721 [0.712,0.728]	+0.000
Random fixed	0.627 [0.595,0.658]	-0.040	0.373 [0.342,0.405]	0.743 [0.733,0.752]	+0.022
Non-diff.	0.580 [0.547,0.612]	-0.087	0.420 [0.388,0.453]	0.757 [0.746,0.767]	+0.036
Blind	0.032 [0.023,0.046]	-0.634	0.968 [0.954,0.977]	0.622 [0.616,0.627]	-0.099

TABLE III

Primary closed-loop navigation performance.

Wilson intervals, fill is the mean valid-depth fraction with bootstrap intervals, and Δ columns are measured relative to the fixed-camera baseline.

The joint improvement in success and fill is the key point. A gain in success without a gain in fill could be explained by a more conservative policy or a different trajectory distribution. A gain in fill without a gain in success would mean that the camera learned to recover pixels that the controller cannot use. The proposed method is the only learned-camera variant that improves both metrics over the fixed baseline. Random-fixed and non-differentiable baselines do increase fill, but they do not improve success. That separation matters: the recovered pixels must be scene-conditioned and must appear at the right part of the aperture to affect collision avoidance. The final policy therefore improves the sensing stream and then converts that improved observation into higher success.

C. Scene-Level Results Show Where the Gain Occurs

The scene breakdown in Table IV reveals that the benefit is not uniform across regimes. Dark material benefits the most because it is fundamentally a return-starvation problem: additional exposure and gain can recover weak returns without erasing the slit boundary, so the extra fill translates cleanly into more successful traversals. Specular scenes are intermediate because the dominant failure mode is local reflection and edge instability around the side-wall patches; the learned camera can damp that instability, but it cannot eliminate it. Glare is the hardest regime because it is a saturation problem near the aperture boundary. The camera can suppress overexposure and recover a large amount of valid depth, yet the remaining error is concentrated exactly where the controller needs clean contrast, so the gain in success is smaller than the gain in fill. Each scene-level entry in Table IV is reported as success/fill, where fill is the mean valid-depth fraction.

This asymmetry is the mechanism-level result. Dark and glare both gain fill, but only dark converts most of that

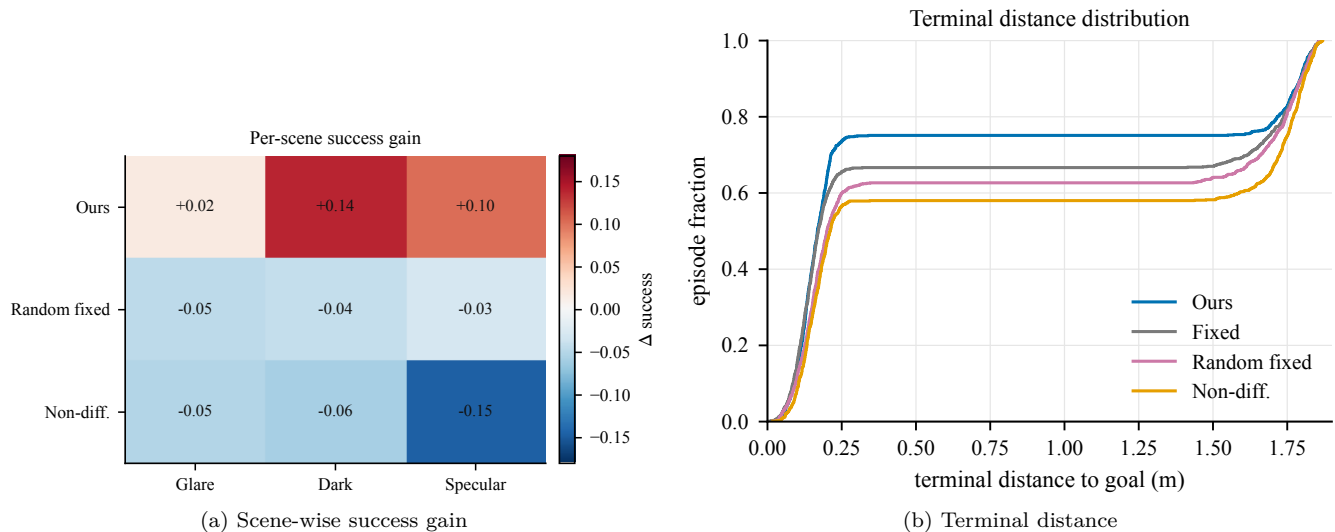


Fig. 4. Closed-loop navigation breakdown. Scene-wise gains are measured relative to the fixed-camera baseline, and terminal-distance distributions show how closely each method approaches the goal over the held-out evaluation set.

Method	Glare	Dark	Specular
	Succ./Fill	Succ./Fill	Succ./Fill
Ours	0.67/0.96	0.82/0.98	0.76/0.99
Fixed	0.65/0.69	0.68/0.70	0.66/0.77
Random fixed	0.61/0.69	0.64/0.73	0.63/0.80
Non-diff.	0.60/0.67	0.62/0.67	0.52/0.93
Blind	0.03/0.60	0.03/0.59	0.03/0.68

TABLE IV
Scene-wise success and depth-fill rates.

gain into success. The difference is physical rather than statistical: dark suppresses return that can be recovered by integration and gain, whereas glare corrupts the contrast at the aperture itself. Fill is therefore necessary but not sufficient; the controller also needs the recovered pixels to preserve the slit boundary.

D. The Learned Camera Implements Scene-Specific Near-Slit Behavior

Table V and Figure 5 test whether the learned camera has interpretable active-sensing semantics. In the near-slit phase, the proposed policy uses high power but low exposure and gain in glare, high exposure and gain in dark material, and a lower-power intermediate setting in specular scenes. The direction of change is physically coherent. In glare, suppressing exposure and gain protects the aperture boundary from saturation. In dark material, increasing exposure and gain compensates for weak return. Specular scenes sit between those extremes, with lower power to avoid reflective hotspots and moderate exposure and gain to stabilize the adjacent patches. The glare-dark separation reaches 0.412 with a narrow confidence interval, whereas fixed camera, non-differentiable learned camera, and blind control remain at or near zero. Random fixed camera has small separation because its sampled settings

differ between episodes, but that separation is not scene-conditional and remains far below the proposed policy. Table V reports normalized near-slit camera commands as $P/E/G$, corresponding to projector power, exposure, and gain. The G-D L1 column is the mean absolute difference between glare and dark near-slit camera commands, with 95% bootstrap confidence intervals.

This analysis is important because a higher fill rate alone would not be enough to claim active sensing. A controller could push all parameters toward one extreme and improve fill in some cases without learning the regime structure. The near-slit fingerprints show the opposite: the policy changes its settings in a scene-conditional way, and the directions match the sensor model. The non-differentiable baseline is revealing because it can raise fill without reproducing the same scene-conditional motion, which explains why its navigation performance remains weaker.

E. Matched-Pose Depth Sequences Isolate the Camera Effect

The qualitative depth sequences in Figures 6–8 provide a visual counterpart to the scalar metrics. Each panel re-renders observed depth at matched poses, using the same raw geometric depth but different camera settings. This matched-pose construction matters because it separates camera effects from trajectory effects. If two methods see different depth images at different poses, the difference could come from either motion or sensing. Here the pose is held fixed, so changes in observed depth are attributable to camera parameters under the same geometry.

The sequences explain why fill alone is not the whole story. In glare, nominal camera settings leave invalid regions around the opening, whereas the learned camera preserves usable aperture structure by suppressing exposure and gain. In dark material, the learned camera

Method	Glare $P/E/G$	Dark $P/E/G$	Specular $P/E/G$	Glare-dark L1 (95% CI)
Ours	0.731/0.088/0.120	0.665/0.694/0.683	0.408/0.366/0.381	0.412 [0.406,0.417]
Fixed	0.500/0.500/0.500	0.500/0.500/0.500	0.500/0.500/0.500	0.000 [0.000,0.000]
Random fixed	0.523/0.522/0.467	0.541/0.521/0.453	0.526/0.504/0.455	0.011 [0.006,0.039]
Non-diff.	0.413/0.373/0.508	0.413/0.373/0.507	0.408/0.365/0.510	0.000 [0.000,0.002]
Pretrain	0.714/0.061/0.103	0.682/0.724/0.709	0.392/0.352/0.365	0.433 [0.427,0.439]
DAgger	0.728/0.069/0.109	0.676/0.727/0.708	0.392/0.360/0.368	0.436 [0.426,0.445]
Blind	0.500/0.500/0.500	0.500/0.500/0.500	0.500/0.500/0.500	0.000 [0.000,0.000]

TABLE V
Near-slit camera commands and glare-dark separation.

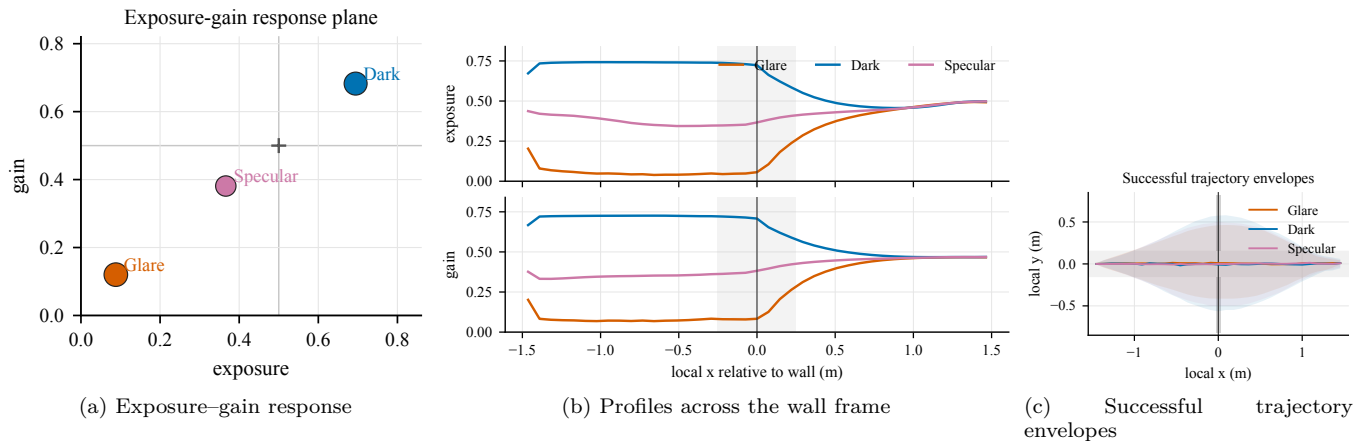


Fig. 5. Near-slit active-camera mechanism. The final policy produces scene-specific camera settings near the wall: glare suppresses exposure and gain, dark material increases them, and specular material uses a distinct lower-power regime. The response plane and wall-frame profiles show how these settings evolve around the aperture, while the trajectory envelope links the sensor behavior to successful slit traversal.

recovers return that would otherwise be weak under nominal settings. In specular material, the gain is more subtle because fixed camera already preserves more geometry, but the active policy still maintains a cleaner observation than the random-static setting. The recovered pixels are therefore not only more numerous; they are better aligned with the aperture and side-wall structures that determine the flight decision.

F. Camera Relabeling and Flight Adaptation Are Complementary

Figure 9 separates two questions that are easy to conflate. The first is whether the camera branch has learned meaningful scene-dependent semantics. The second is whether the flight branch can use the resulting observation distribution to navigate. Camera pretraining and DAgger relabeling answer the first question. They recover clear glare-dark camera separation in online rollouts, with high exposure and gain in dark material and low exposure and gain in glare. However, these camera-only policies do not answer the second question: they produce meaningful camera semantics, but the flight controller has not yet adapted to the resulting observation stream.

This is expected. A learned camera changes the distribution of depth images seen by the controller. Even if the camera is semantically correct, the flight policy still has to adapt to the new observation stream. The final flight-only stage performs that adaptation while freezing the

camera branch. The result is that the final policy preserves the DAgger-learned camera separation while achieving substantially higher navigation success than the camera-only diagnostic policies. The diagnostic checkpoints are therefore useful rather than contradictory: they show that camera semantics can be learned before they become operationally useful for closed-loop flight.

VI. Discussion

A. What the Evidence Supports

The experiments support a focused claim: in this controlled slit-navigation benchmark, task-aware active depth sensing improves closed-loop flight when the camera semantics learned from a differentiable teacher are preserved during final policy adaptation. The evidence is not based on a single metric. Success improves relative to the primary baselines, collision decreases, depth fill increases substantially, near-slit camera settings separate according to scene regime, and matched-pose renderings show that those settings change the observed depth image under the same geometry.

The regime asymmetry is the most informative part of that result. Dark scenes improve the most in success because they are return-starvation problems: additional exposure and gain recover weak returns without destroying the slit boundary. Glare improves more in fill than in success because it is a saturation problem: the camera

Observed depth at matched poses: glare, slot=far_right

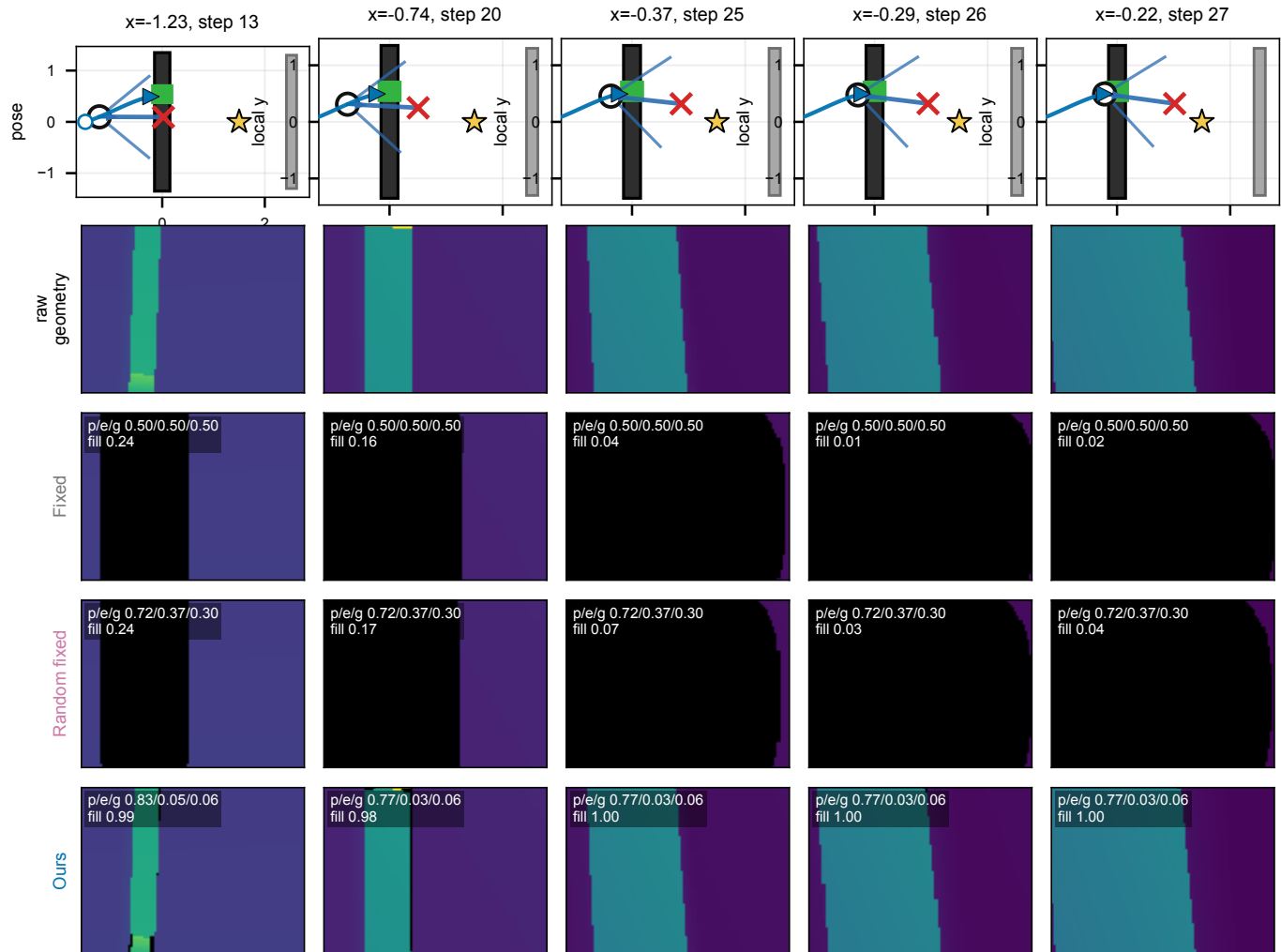


Fig. 6. Matched-pose depth observation sequences rendered along the same rollout poses (Glare). The raw geometry is fixed; only the camera parameters change the observed depth.

can recover valid pixels, but the aperture edge is still vulnerable to contamination. Specular scenes sit between those extremes because local reflection creates holes and edge drift that the camera can mitigate but not completely remove. The benchmark therefore shows not just that active sensing helps, but that it helps for physically meaningful reasons.

The split-stem design is important for this conclusion. The camera branch is not merely another head attached to a representation that continues to move during flight training. It has its own copied visual pathway and camera-specific modules, and these components are frozen during flight-only adaptation. This keeps the meaning of the camera head stable while allowing the flight branch to adapt to the learned-camera depth stream.

B. What the Evidence Does Not Claim

The result should not be read as a claim that active camera control solves general aerial navigation or that depth fill alone determines success. The benchmark is intentionally narrow, and the strongest navigation gain occurs in the dark-material regime. Glare remains difficult: the active camera improves fill substantially, but success improves only modestly. This distinction is important. A valid depth pixel is useful only if it preserves the task-critical aperture cue at the right time and location. Global fill is therefore a necessary but incomplete indicator of navigation quality. The random-fixed and non-differentiable baselines make the same point from the negative side: they can raise fill, but if the recovered pixels are not scene-conditioned, the navigation policy does not improve.

Nor should the pretrain and DAgger diagnostic policies be interpreted as failed camera controllers. Their purpose

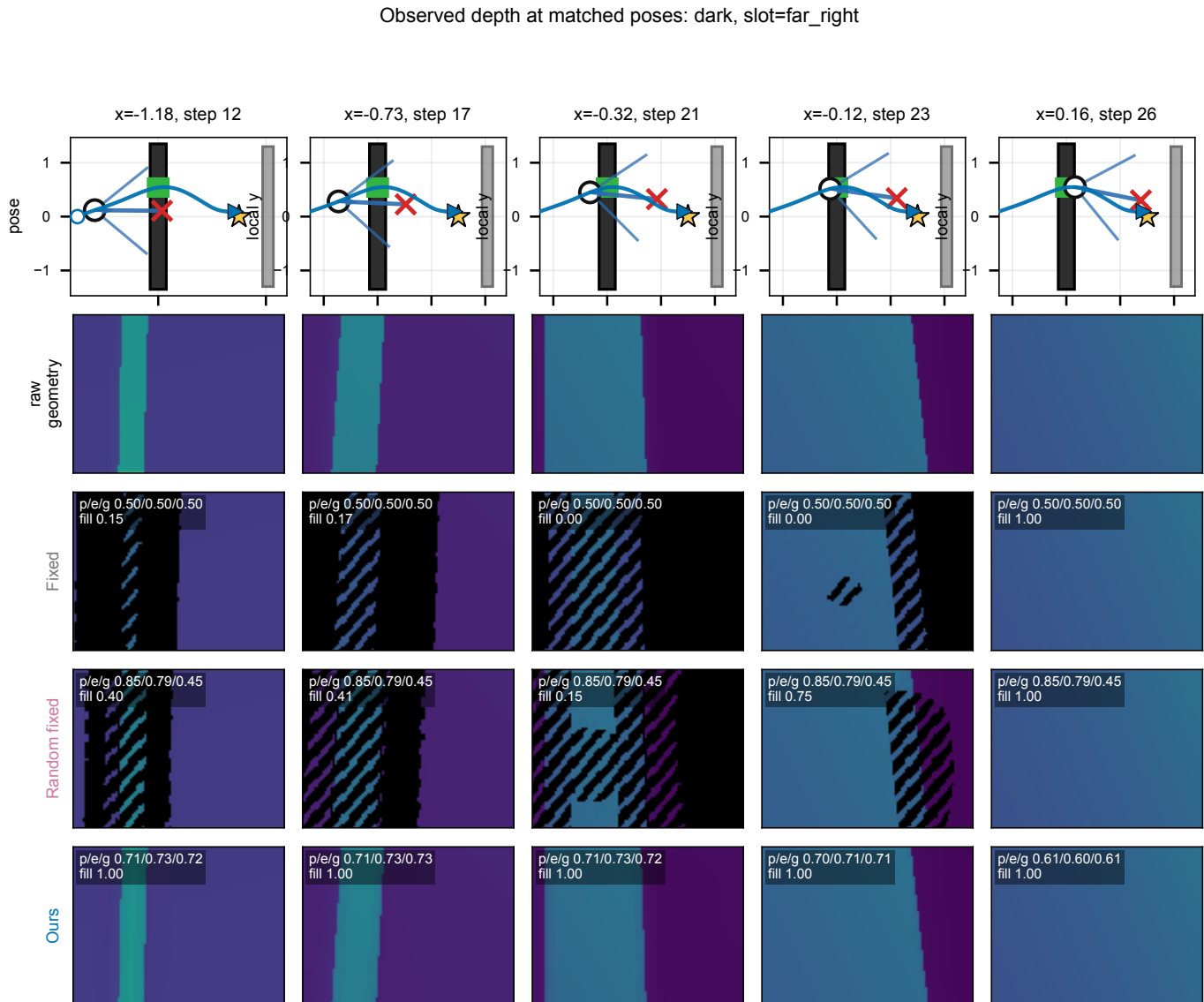


Fig. 7. Matched-pose depth observation sequences rendered along the same rollout poses (Dark). The raw geometry is fixed; only the camera parameters change the observed depth.

is to show that camera semantics can be learned and preserved. Their low navigation success demonstrates a different point: the flight controller must be adapted after the camera changes the closed-loop observation distribution.

C. Limitations and Threats to Validity

The study is simulation-only. The sensor model is designed to preserve the causal relationships among power, exposure, gain, motion blur, noise, and localized depth degradation, but it is not a pixel-accurate model of any specific commercial depth camera. Transferring the method to hardware would require calibration of the sensor parameters, validation under real lighting and material conditions, and safeguards for register limits and temporal camera latency.

The evaluation also uses a single training seed. The held-out episode count is large enough to estimate closed-loop performance for the trained checkpoints, but it does not measure variance across independently trained policies. A stronger future study would report multiple training seeds, additional slit widths, and hardware-in-the-loop or real-robot trials. These extensions are outside the scope of the present benchmark, whose purpose is to isolate the mechanism of active depth sensing under controlled degradation.

VII. Conclusion

We presented a split-stem active depth sensing pipeline for quadrotor slit navigation. The method treats camera parameters as part of the closed loop, learns camera behavior from differentiable teacher relabeling, and then adapts the flight branch while freezing the camera branch.

Observed depth at matched poses: specular, slot=far_right

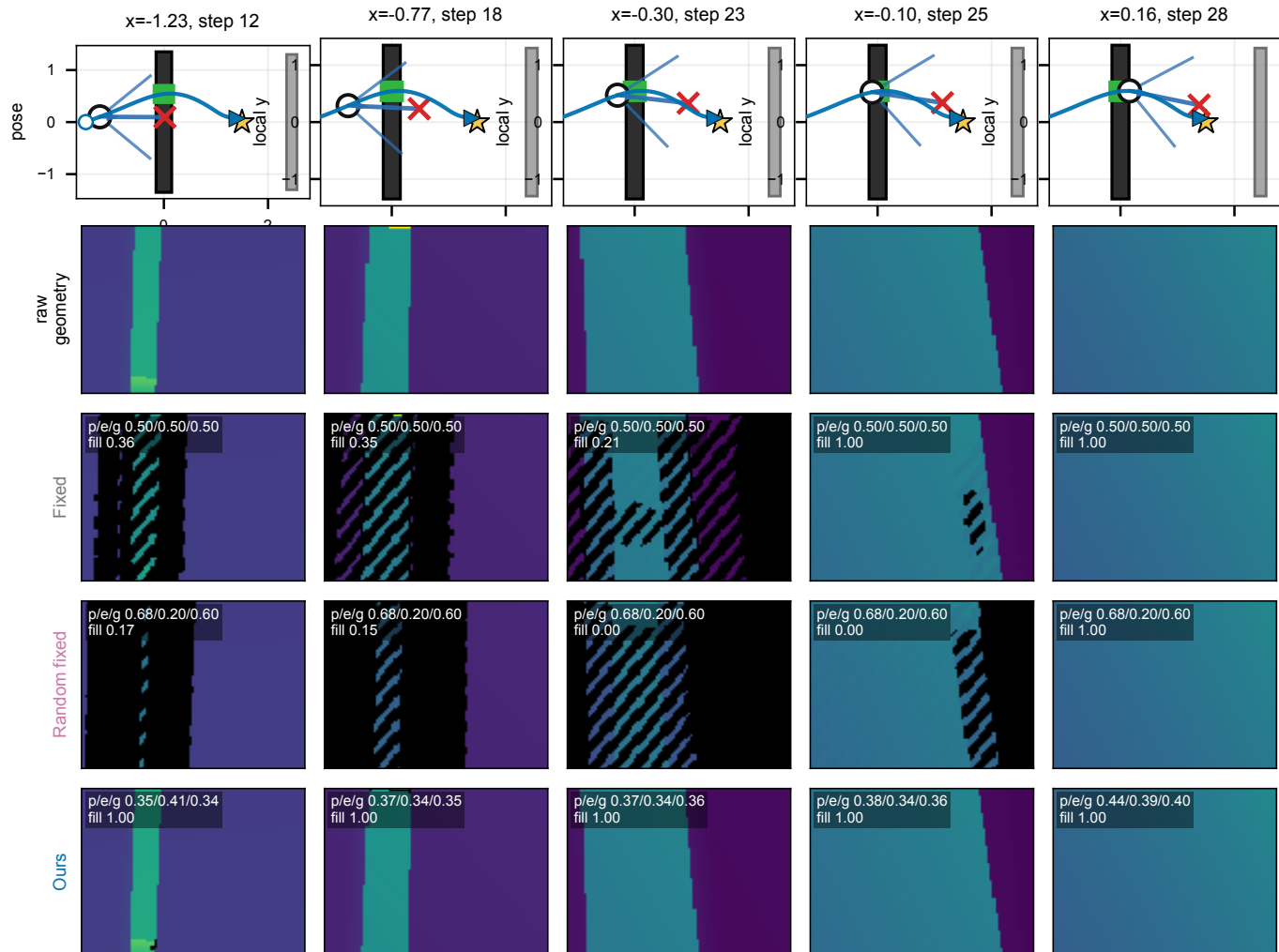


Fig. 8. Matched-pose depth observation sequences rendered along the same rollout poses (Specular). The raw geometry is fixed; only the camera parameters change the observed depth.

In held-out evaluation, the final policy improves success and depth fill over fixed, random-fixed, non-differentiable learned-camera, and blind baselines. The mechanism analysis shows that this improvement is accompanied by interpretable scene-specific camera behavior near the slit. These results indicate that active depth control is most effective when sensing semantics and flight adaptation are treated as coupled but separately stabilized parts of the policy.

References

- [1] R. Newbury, J. Collins, K. He, J. Pan, I. Posner, D. Howard, and A. Cosgun, “A Review of Differentiable Simulators,” Jul. 2024.
- [2] X. Zhang, R. Wang, Y. Ren, J. Sun, H. Fang, J. Chen, and G. Wang, “DiffAero: A GPU-Accelerated Differentiable Simulation Framework for Efficient Quadrotor Policy Learning,” Sep. 2025.
- [3] Y. Zhang, Y. Hu, Y. Song, D. Zou, and W. Lin, “Learning vision-based agile flight via differentiable physics,” *Nature Machine Intelligence*, vol. 7, no. 6, pp. 954–966, Jun. 2025.
- [4] J. Heeg, Y. Song, and D. Scaramuzza, “Learning Quadrotor Control from Visual Features Using Differentiable Simulation,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, May 2025, pp. 4033–4039.
- [5] B.-H. Chen, N. M. Batagoda, and D. Negrut, “DiffPhysCam: Differentiable Physics-Based Camera Simulation for Inverse Rendering and Embodied AI,” Aug. 2025.
- [6] P. Florence, J. Carter, and R. Tedrake, “Integrated Perception and Control at High Speed: Evaluating Collision Avoidance Maneuvers Without Maps,” in *Algorithmic Foundations of Robotics XII*, K. Goldberg, P. Abbeel, K. Bekris, and L. Miller, Eds. Cham: Springer International Publishing, 2020, vol. 13, pp. 304–319.
- [7] Y. Song, K. Shi, R. Penicka, and D. Scaramuzza, “Learning Perception-Aware Agile Flight in Cluttered Environments,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, May 2023, pp. 1989–1995.
- [8] J. Tordesillas and J. P. How, “Deep-PANTHER: Learning-Based Perception-Aware Trajectory Planner in Dynamic Environments,” *IEEE Robotics and Automation Letters*, vol. 8,

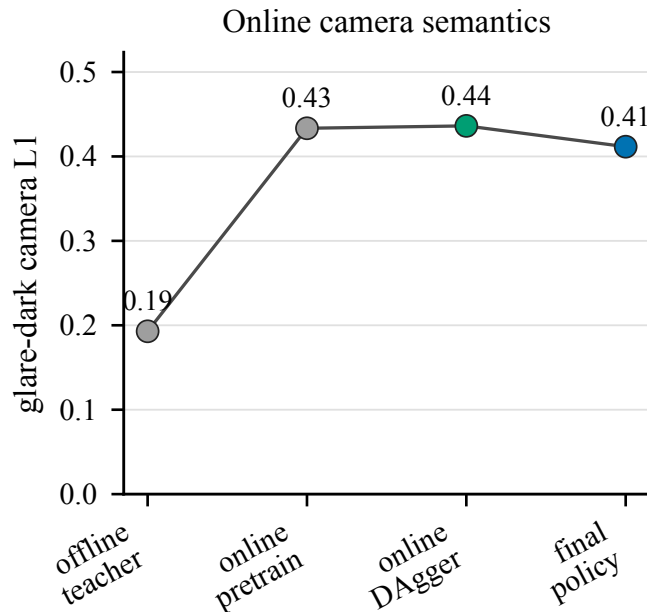


Fig. 9. Camera relabeling restores scene-specific camera semantics. The glare-dark camera separation appears in the relabeled teacher data and is preserved through the final split-stem policy.

- no. 3, pp. 1399–1406, Mar. 2023.
- [9] G. Zhao, T. Wu, Y. Chen, and F. Gao, “Learning Speed Adaptation for Flight in Clutter,” *IEEE Robotics and Automation Letters*, vol. 9, no. 8, pp. 7222–7229, Aug. 2024.
 - [10] Y. Hu, Y. Zhang, Y. Song, Y. Deng, F. Yu, L. Zhang, W. Lin, D. Zou, and W. Yu, “Seeing Through Pixel Motion: Learning Obstacle Avoidance From Optical Flow With One Camera,” *IEEE Robotics and Automation Letters*, vol. 10, no. 6, pp. 5871–5878, Jun. 2025.
 - [11] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, 2011, pp. 627–635.
 - [12] A. Romero, E. Aljalbout, Y. Song, and D. Scaramuzza, “Actor-Critic Model Predictive Control: Differentiable Optimization meets Reinforcement Learning,” Feb. 2025.
 - [13] M. Denninger, D. Winkelbauer, M. Sundermeyer, W. Boerdijk, M. Knauer, K. H. Strobl, M. Humt, and R. Triebel, “Blender-Proc2: A procedural pipeline for photorealistic rendering,” in *Journal of Open Source Software*, vol. 8, no. 82, 2023, p. 4901.
 - [14] A. Amini, T.-H. Wang, I. Gilitschenski, W. Schwarting, Z. Liu, S. Han, S. Karaman, and D. Rus, “VISTA 2.0: An open, data-driven simulator for multimodal sensing and policy learning for autonomous vehicles,” in *2022 IEEE International Conference on Robotics and Automation (ICRA)*, 2022, pp. 2419–2426.
 - [15] X. Puig, E. Undersander, A. Szot, M. Dallaire Cote, T.-Y. Yang, R. Partsey, R. Desai, A. W. Clegg, M. Hlavac, S. Y. Min, V. Vondrus, T. Gervet, V.-P. Berges, J. M. Turner, O. Maksymets, Z. Kira, M. Kalakrishnan, J. Malik, D. S. Chaplot, U. Jain, D. Batra, A. Rai, and R. Mottaghi, “Habitat 3.0: A co-habitat for humans, avatars and robots,” 2023.
 - [16] A. Elmquist, R. Serban, and D. Negrut, “A sensor simulation framework for training and testing robots and autonomous vehicles,” *Journal of Autonomous Vehicles and Systems*, vol. 1, no. 2, p. 021001, 2021.
 - [17] H. Blasinski, J. Farrell, T. Lian, Z. Liu, and B. Wandell, “Optimizing image acquisition systems for autonomous driving,” *Electronic Imaging*, vol. 2018, no. 5, pp. 161–1–161–7, 2018.
 - [18] Z. Liu, T. Lian, J. Farrell, and B. A. Wandell, “Neural network generalization: The impact of camera parameters,” *IEEE Access*, vol. 8, pp. 10 443–10 454, 2020.
 - [19] Z. Lyu, T. Goossens, B. A. Wandell, and J. Farrell, “Validation of physics-based image systems simulation with 3d scenes,” *IEEE Sensors Journal*, vol. 22, no. 20, pp. 19 400–19 410, 2022.
 - [20] A. Carlson, K. A. Skinner, R. Vasudevan, and M. Johnson-Roberson, “Modeling camera effects to improve visual learning from synthetic data,” in *European Conference on Computer Vision Workshops*, 2018.
 - [21] —, “Sensor transfer: Learning optimal sensor effect image augmentation for sim-to-real domain adaptation,” *IEEE Robotics and Automation Letters*, vol. 4, no. 3, pp. 2431–2438, 2019.
 - [22] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, “NeRF: Representing scenes as neural radiance fields for view synthesis,” in *European Conference on Computer Vision*, 2020, pp. 405–421.
 - [23] T. Müller, A. Evans, C. Schied, and A. Keller, “Instant neural graphics primitives with a multiresolution hash encoding,” *ACM Transactions on Graphics*, vol. 41, no. 4, pp. 1–15, 2022.
 - [24] M. Tancik, E. Weber, E. Ng, R. Li, B. Yi, T. Wang, A. Kristoffersen, J. Austin, K. Salahi, A. Ahuja, D. McAllister, J. Kerr, and A. Kanazawa, “Nerfstudio: A modular framework for neural radiance field development,” in *ACM SIGGRAPH Conference Proceedings*, 2023.
 - [25] B. Kerbl, G. Kopanas, T. Leimkuehler, and G. Drettakis, “3d gaussian splatting for real-time radiance field rendering,” *ACM Transactions on Graphics*, vol. 42, no. 4, pp. 1–14, 2023.
 - [26] A. Guédon and V. Lepetit, “SuGaR: Surface-aligned gaussian splatting for efficient 3d mesh reconstruction and high-quality mesh rendering,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 5354–5363.
 - [27] J. Hasselgren, N. Hofmann, and J. Munkberg, “Shape, light, and material decomposition from images using monte carlo rendering and denoising,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 22 856–22 869.
 - [28] H. Ouyang, Z. Shi, C. Lei, K. L. Law, and Q. Chen, “Neural camera simulators,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 7696–7705.
 - [29] S. Pidhorskyi, T. Bagautdinov, S. Ma, J. Saragih, G. Schwartz, Y. Sheikh, and T. Simon, “Depth of field aware differentiable rendering,” *ACM Transactions on Graphics*, vol. 41, no. 6, 2022.
 - [30] C. Sun, Y. Li, J. Li, C. Wang, and X. Dai, “CaSE-NeRF: Camera settings editing of neural radiance fields,” in *Computer Graphics International*, 2023, pp. 95–107.
 - [31] M.-J. Kim, G. Gu, and J. Choo, “LensNeRF: Rethinking volume rendering based on thin-lens camera model,” in *IEEE/CVF Winter Conference on Applications of Computer Vision*, 2024, pp. 3182–3191.

- [32] Z. Wu, X. Li, J. Peng, H. Lu, Z. Cao, and W. Zhong, “DoF-NeRF: Depth-of-field meets neural radiance fields,” in *ACM International Conference on Multimedia*, 2022, pp. 1718–1729.
- [33] Y. Wang, S. Yang, Y. Hu, and J. Zhang, “NeRFocus: Neural radiance field for 3d synthetic defocus,” 2022.
- [34] R. Bajcsy, “Active perception,” *Proceedings of the IEEE*, vol. 76, no. 8, pp. 996–1005, 1988.
- [35] J. Aloimonos, I. Weiss, and A. Bandyopadhyay, “Active vision,” *International Journal of Computer Vision*, vol. 1, no. 4, pp. 333–356, 1988.
- [36] R. Bajcsy, J. Aloimonos, and J. K. Tsotsos, “Revisiting active perception,” *Autonomous Robots*, vol. 42, no. 2, pp. 177–196, 2018.
- [37] A. Loquercio, A. I. Maqueda, C. R. del Blanco, and D. Scaramuzza, “Deep drone racing: From simulation to reality with domain randomization,” *IEEE Robotics and Automation Letters*, vol. 3, no. 1, pp. 133–140, 2018.
- [38] A. Loquercio, E. Kaufmann, R. Ranftl, M. Müller, V. Koltun, and D. Scaramuzza, “Learning high-speed flight in the wild,” *Science Robotics*, vol. 6, no. 59, p. eabh5217, 2021.
- [39] P. Foehn, E. Kaufmann, A. Romero, R. Penicka, S. Sun, L. Bauersfeld, T. Laengle, G. Cioffi, Y. Song, A. Loquercio, and D. Scaramuzza, “Agilicious: Open-source and open-hardware agile quadrotor for vision-based flight,” *Science Robotics*, vol. 7, no. 67, p. eabl6259, 2022.
- [40] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, “Champion-level drone racing using deep reinforcement learning,” *Nature*, vol. 620, pp. 982–987, 2023.